PLC Programming Languages Laboratory Report

Course: Intro to Critical Infrastructure for IT Learners

Lab Title: Lab 02 — PLC Programming Languages

Student Name: Joseph Agyapong

Student ID: C1126ACISA17354

Target VM IP: 192.168.1.10

Date: September 24, 2026

Executive Summary

This laboratory report proves the possibility of using the five standard IEC 61131-3 PLC programming languages. Specifically, the Ladder Diagram (LD), Function Block Diagram (FBD), Structured Text (ST), Instruction List (IL), and Sequential Function Chart (SFC) programming languages are implemented in the OpenPLC Editor to control a standard motor start/stop holding (seal-in) circuit. This report contains definitions of system variables, evidence placeholders for each step, explanation notes, a comparison table, and answers to assessment questions.

1. System Overview & Variable Identification

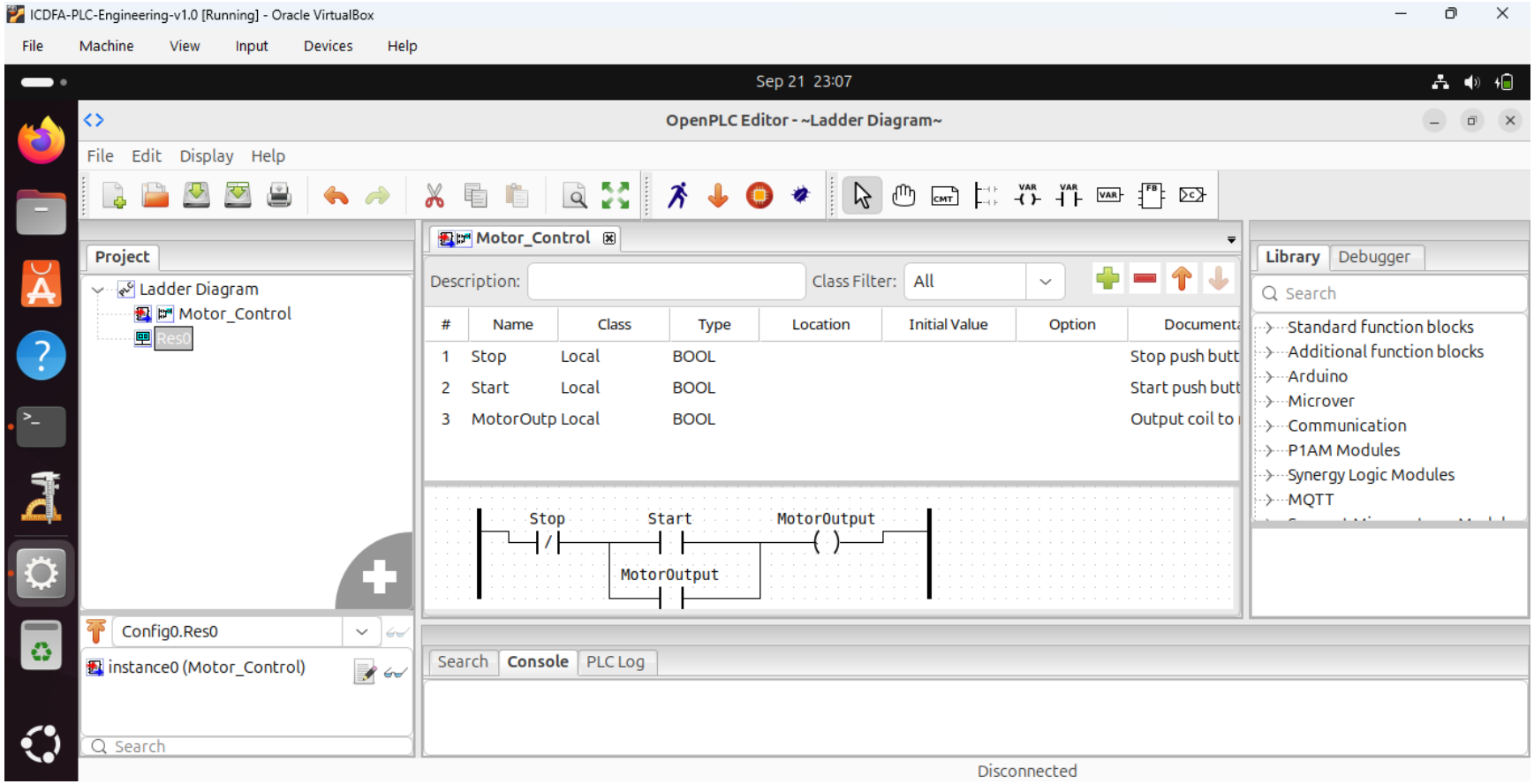
Industrial automation relies on standardized tags and discrete Boolean data types (BOOL, evaluating to 0/FALSE or 1/TRUE). The motor start/stop control uses a **holding (seal-in) circuit** to ensure the motor remains energized after a momentary Start button is released, until a momentary Stop button is pressed.

Tag / Variable Table (Step 4 Requirement)

Tag Name	Data Type	Scope / Address	Functional Purpose JPG
Start	BOOL	Local / Input	Momentary normally open push-button input used to initiate the motor run sequence.
Stop	BOOL	Local / Input	Momentary normally closed push-button input used to interrupt power/logic and stop the motor.
MotorOutput (or MotorOutp)	BOOL	Local / Output	Output coil controlling the physical motor starter; provides holding/latching logic feedback.

2. Practical Implementations & Evidence

Evidence Item 1: Ladder Diagram (LD) Implementation

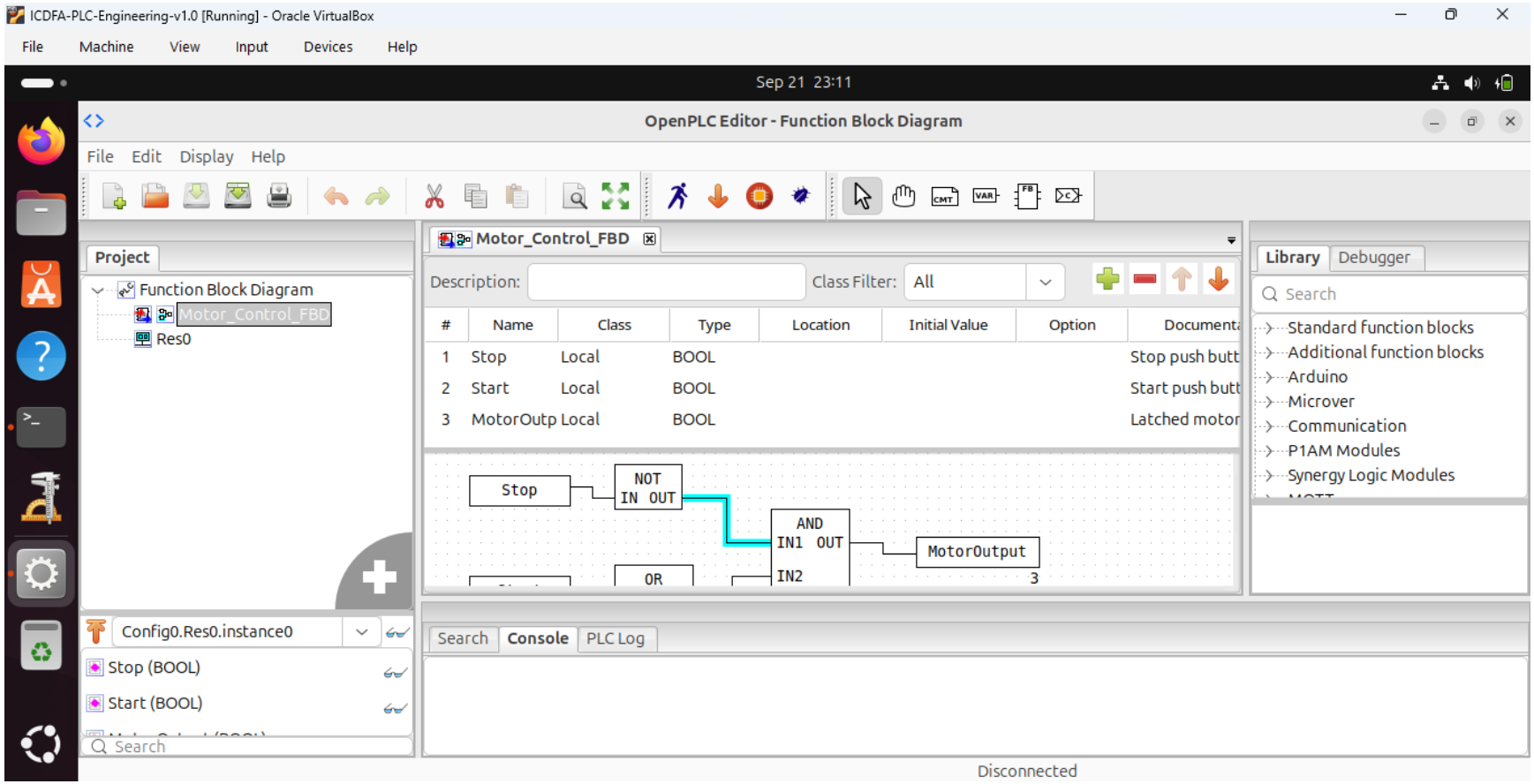


(Shows OpenPLC Editor with Motor_Control tab, variable table, and the LD rung containing Start, Stop [Negated], and MotorOutput).

- **Technical Explanation:**
  - **Start Contact (| |):** Normally Open contact that closes when Start is activated (TRUE).

- **Stop Contact (| / |):** Negated / Normally Closed contact that allows logic continuity until Stop is pressed (TRUE), which breaks the circuit.
- **MotorOutput Latch Contact (| | in parallel):** Connected in parallel across Start. Once MotorOutput energizes, this contact maintains power flow around Start after it is released.
- **MotorOutput Coil (( )):** Output coil energized when a continuous Boolean logic path exists across the rung.

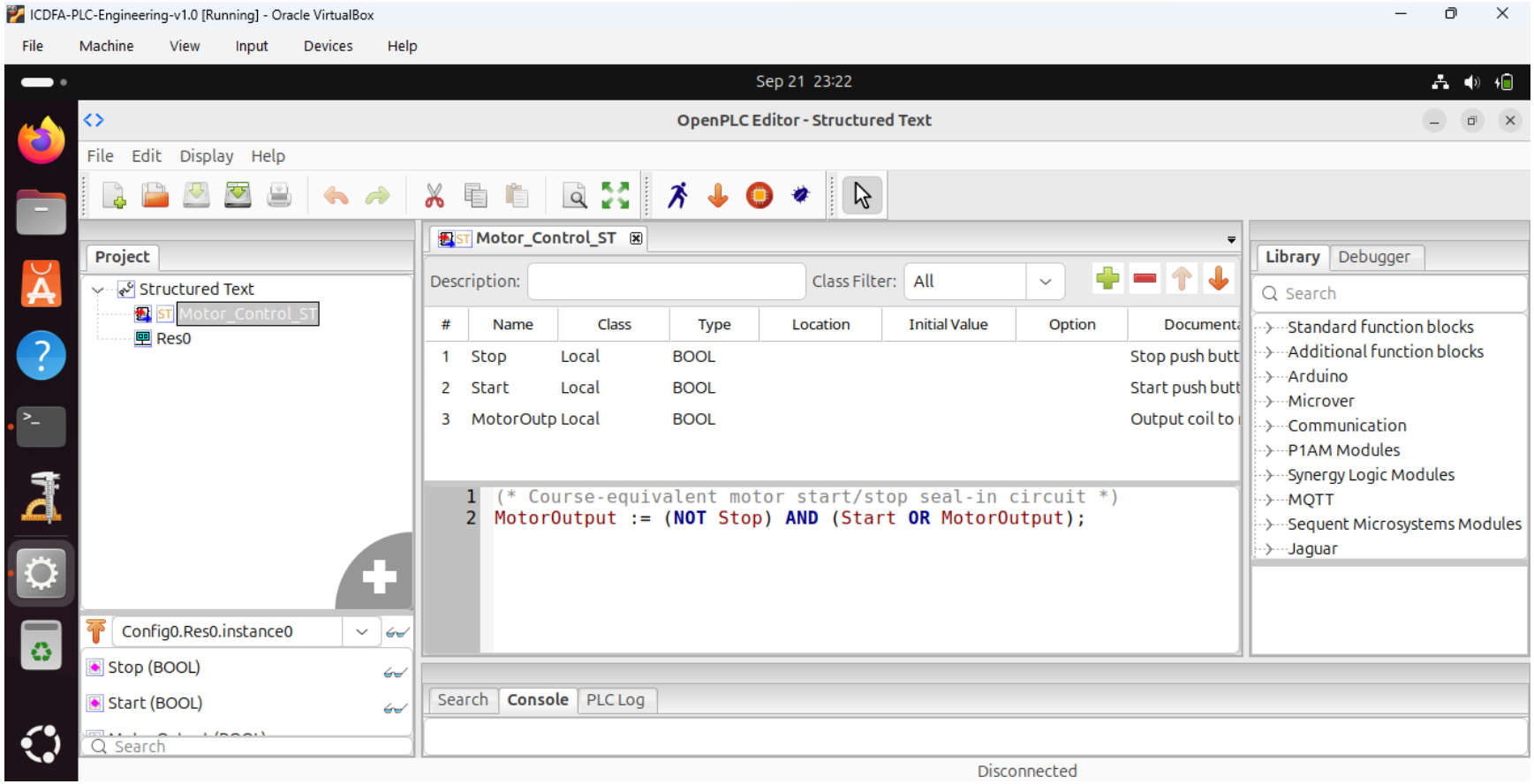
Evidence Item 2: Function Block Diagram (FBD) Implementation



(Shows OpenPLC Editor with Motor_Control_FBD tab, variable table, and OR, NOT, AND graphical blocks).

- **Technical Explanation:**
    - **OR Block:** Evaluates Start OR MotorOutput, creating the software equivalent of the parallel holding contact.
    - **NOT Block:** Inverts the Stop signal (NOT Stop), keeping the interlock high during normal operation.
    - **AND Block:** Combines the outputs of the OR block and NOT block ((Start OR MotorOutput) AND NOT Stop) to drive MotorOutput.

Evidence Item 3: Structured Text (ST) Implementation



(Shows OpenPLC Editor with Motor_Control_ST tab, variable table, and the textual assignment code).

- **Textual Code Statement:**

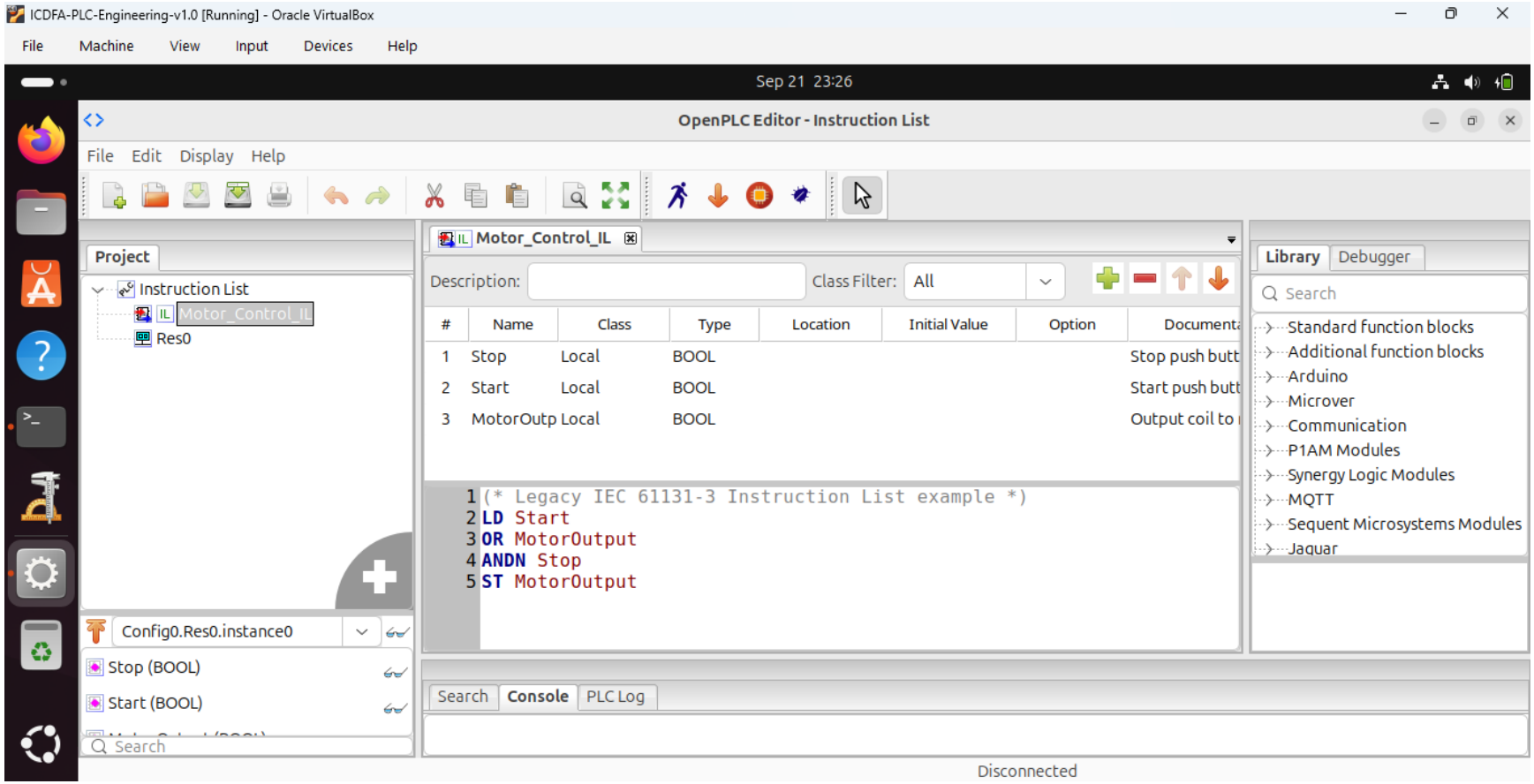
Delphi

```
(* Structured Text Motor Start/Stop Seal-in Circuit *)

MotorOutput := (NOT Stop) AND (Start OR MotorOutput);
```

- **Technical Explanation:**
    - Uses high-level, procedural syntax evaluated cyclically by the PLC scan engine.
    - The expression evaluates whether Stop is unpressed (NOT Stop = TRUE) while either Start is active or MotorOutput is already latched (Start OR MotorOutput).
    - The resulting Boolean state is assigned directly to MotorOutput via the assignment operator :=.

Evidence Item 4: Instruction List (IL) Implementation



(Shows OpenPLC Editor with Motor_Control_IL tab, variable table, and sequential assembly instructions).

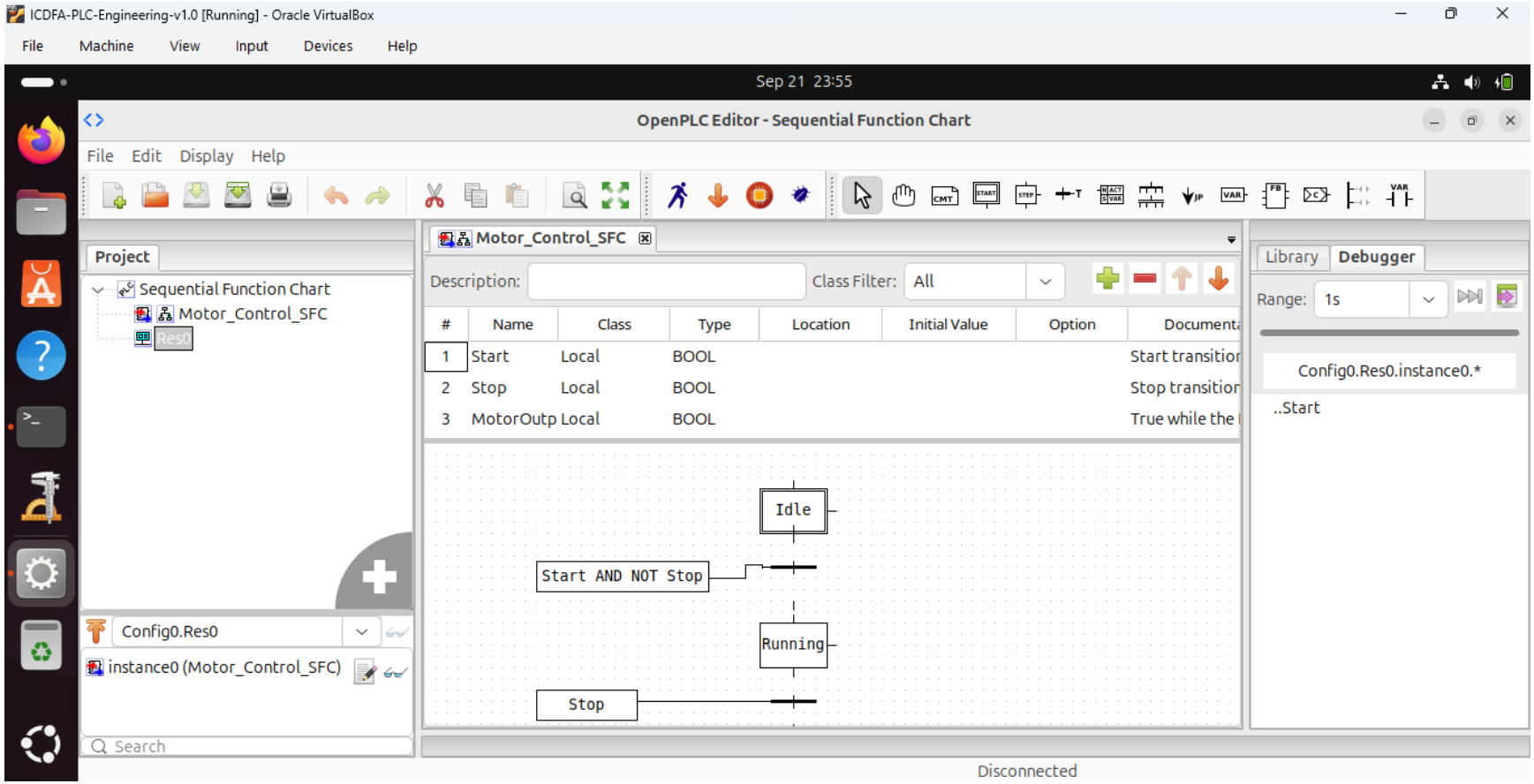
- **Textual Mnemonic Instructions:**

Code snippet

```
LD    Start          (* Load Start contact status into accumulator *)
OR    MotorOutput    (* Perform logical OR with MotorOutput status *)
ANDN  Stop           (* Perform logical AND NOT with Stop contact *)
ST    MotorOutput    (* Store result in MotorOutput coil *)
```

- **Technical Explanation:**
    - Low-level assembly-like language operating via an internal result register/accumulator.
    - Executes strictly line-by-line: loads the Start state, applies an OR operation with MotorOutput to sustain the seal-in, applies an AND NOT operation with Stop, and stores (ST) the outcome to the output tag.

Evidence Item 5: Sequential Function Chart (SFC) Implementation



(Shows OpenPLC Editor with Motor_Control_SFC tab, variable table, Idle/Running steps, and transition conditions).

- **Technical Explanation:**
  - **Idle Step:** Initial double-bordered step representing the system in its resting state (Motor OFF).

- **Transition 1 (Start AND NOT Stop):** Guard condition that validates the start command and evaluates whether it is safe to transition from Idle to Running.
- **Running Step:** Active operational step associated with the action that energizes MotorOutput := TRUE.
- **Transition 2 (Stop):** Condition that triggers state exit from Running and loops execution back to Idle.

3. Comparative Analysis Matrix (Step 9 Requirement)

Language	Representation Type JPG	Core Strengths JPG	Limitations JPG	Primary Industrial Use Case JPG
Ladder Diagram (LD)	Graphical (Relay Rung Logic)	Intuitive for electrical technicians; easy real-time visual power-flow troubleshooting.	Cluttered and inefficient for complex mathematical calculations or algorithms.	Discrete relay circuits, safety interlocks, motor start/stop controls.
Function Block Diagram (FBD)	Graphical (Signal Flow Blocks)	Excellent visual signal tracking; highly modular and reusable control blocks.	Diagram can become crowded and difficult to follow in large sequential state applications.	Analog process control loops (PID), safety instrumented systems (SIS).
Structured Text (ST)	Textual (High-Level Procedural)	Compact code; ideal for complex math, conditional logic	Harder for field technicians without software	Complex mathematical algorithms, data parsing,

Language	Representation Type JPG	Core Strengths JPG	Limitations JPG	Primary Industrial Use Case JPG
		(IF/THEN), loops, and arrays.	backgrounds to debug live on physical hardware.	communication drivers, recipe management.
Instruction List (IL)	Textual (Low-Level Assembly)	High execution speed; low computational and memory footprint.	Difficult to maintain; prone to syntax/logic bugs; deprecated in IEC 61131-3 3rd edition.	Legacy micro-PLCs, execution-critical low-level driver subroutines.
Sequential Function Chart (SFC)	Graphical (State Machine Flowchart)	Clear top-level visual sequencing; decomposes complex multi-stage processes into discrete steps.	Requires underlying languages (LD/ST/FBD) to perform specific step actions.	Multi-stage batch processing, packaging lines, sequential state-machine control.

4. Knowledge Check Responses

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: Ladder Diagram is the relay logic language that directly resembles real electromechanical relay schematics. The Ladder Diagram uses the visualized vertical power rails, together with standard symbols for contacts (NO, NC) and coils that are wired through real physical relays and contactors. This notation is understandable by maintenance personnel because it directly mirrors the electrical schematic drawings of electromechanical relay systems, which use the same symbols and power rails for wiring diagram purposes.

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: Structured Text offers the capability to represent high-level mathematical formulations, utilize variable indexes, create arrays, and apply control loops (IF-THEN-ELSE, FOR, WHILE). In contrast, graphical languages such as LD or FBD require a significant amount of blocks or rungs to express a single mathematical expression or a complex algorithm with multiple variables.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: A Sequential Function Chart is a better choice than a continuous ladder logic diagram for controlling a multi-stage industrial process (like a batch chemical process: fill, heat, mix, drain), since an SFC provides a clearer view of the overall process by breaking it into individual functions, transitions between functions, and actions associated with transitions, avoiding the "ladder rung logic" messiness and race condition opportunities of trying to encode everything as a single state machine**.** .

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: Tags ensure that Boolean variables are mapped to physical field inputs, internal memory offsets, and hardware output actuators. Maintaining consistent tag definitions in all programming representations ensures deterministic control, avoids dangerous physical hardware states, and ensures that converting logic between languages does not alter machine safety or control behavior.